

Digital Signal Processing and Analysis (DSPA)
LABORATORY MANUAL
VI Semester (22EI62)
(2025-2026)



DAYANANDA SAGAR COLLEGE OF ENGINEERING
SHAVIGE MALLESWARA HILLS, KUMARASWAMY LAYOUT
BENGALURU-560078

Accredited by National Assessment & Accreditation Council (NAAC) with 'A' Grade
(An Autonomous Institution affiliated to Visvesvaraya Technological University, Belagavi
&
ISO 9001:2015 Certified)

ELECTRONICS & INSTRUMENTATION ENGINEERING DEPARTMENT
(Accredited By NBA)

Vision of the Institute

To impart quality technical education with a focus on Research and Innovation emphasizing on Development of Sustainable and Inclusive Technology for the benefit of society.

Mission of the Institute

- To provide an environment that enhances creativity and Innovation in pursuit of Excellence.
- To nurture teamwork in order to transform individuals as responsible leaders and entrepreneurs.
- To train the students to the changing technical scenario and make them to understand the importance of Sustainable and Inclusive technologies.



Digital Signal Processing and Analysis (DSPA)
LABORATORY MANUAL
(2025-2026)

Name of the Student:

Semester:

USN :

:

DAYANANDA SAGAR COLLEGE OF ENGINEERING
(An Autonomous Institution affiliated to Visvesvaraya Technological University, Belagavi)
DEPARTMENT OF ELECTRONICS & INSTRUMENTATION ENGINEERING
(Accredited By NBA)

BENGALURU-560078

VISION OF THE DEPARTMENT

To meet the challenges of industry and research in the field of Electronics, Instrumentation and Control Engineering by providing quality technical education

MISSION OF THE DEPARTMENT

- To impart quality education with an understanding of basic concepts
- To enhance the knowledge in the core domain of Electronics & Instrumentation by providing a conducive learning environment
- To nurture the students with inter - disciplinary technology by contributing solutions for techno-social issues

PROGRAMME EDUCATIONAL OBJECTIVES [PEOs]

- **PEO1:** Graduates will have the successful professional careers in the core & allied domains
- **PEO2:** Graduates will be able to engage in continuous learning and adapt to modern Technology
- **PEO3:** Graduates will be able to work in diverse environments with team work and Leadership quality
- **PEO4:** Graduates will be able to analyze and provide the solutions to real-life engineering Problems for the betterment of society.

PROGRAMME SPECIFIC OUTCOMES [PSOs]

PSO-1: Apply the knowledge of measurement, control and automation to design and develop the industrial control system for Process Industries.

PSO-2: Acquire the concepts of embedded systems and apply them to solve the engineering problems.

PSO-3: Ability to design instrumentation systems to solve real time applications.

PROGRAM OUTCOMES (POs)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

DAYANANDA SAGAR COLLEGE OF ENGINEERING, BENGALURU-560078
(An Autonomous Institution affiliated to Visvesvaraya Technological University, Belagavi)
DEPARTMENT OF ELECTRONICS & INSTRUMENTATION ENGINEERING
(Accredited By NBA)

DSPA LABORATORY (IPCC)

(SYLLABUS)

VI SEMESTER B. E (EIE)

Course Code: 22 EI62

L: P: T: S: 3: 0: 2: 0

Exam Hours: 02

Credits: 01

CIE Marks: 50

SEE Marks: 50

Course Objectives:

1. To understand and analyze the classification of Signals and their properties.
2. Find the Linear system response convolution sum and Convolution Integral.
3. To understand and analyze the fundamental concepts and properties of DFT and FFT algorithms
4. Design and Analyze the response of different types of FIR and IIR filters

Exp No	Experiment Name	Hours	(CO)
1	Develop the Program to generate continuous waveforms and perform basic operations on signals on MATLAB IDE.	2	CO1
2	Develop the Program to generate discrete waveforms and perform basic operations on signals on MATLAB IDE.	2	CO1
3	Develop the program to determine linear convolution using MATLAB IDE and CC studio IDE.	2	CO1,CO2
4	Develop the program to determine circular convolution of two given discrete time sequences Using MATLAB IDE and CC Studio IDE.	2	CO1,CO2
5	Develop the program to verify Sampling theorem using MATLAB IDE and CC studio IDE.	2	CO1,CO2
6	Develop the program for N point DFT of a given discrete time sequence and to plot magnitude and phase spectrum using MATLAB IDE and CC Studio IDE	2	CO1,CO3
7	Develop a program to determine linear convolution of two given discrete sequences using DFT and IDFT algorithms on MATLAB IDE.	2	CO1,CO3
8	Develop a program to determine circular convolution of two given discrete time sequences using DFT and IDFT algorithms on MATLAB IDE.	2	CO1,CO3
9	Design and Develop the Program for FIR filter response using Rectangular and Hamming Window Technique using MATLAB IDE and CC studio	2	CO1,CO4
10	Design and Develop the Program for Butterworth IIR filter response using MATLAB IDE and CC studio.	2	CO1,CO4

Course Outcomes:

After completion of the course, Students are able to

CO1	Comprehend the fundamentals of signals and systems and perform different mathematical operations on continuous and discrete time signals.
CO2	Analyze the Linear systems using the convolution sum and convolution integral.
CO3	Determine the DFT, IDFT and FFT of given discrete time signals.
CO4	Design and implement IIR and FIR digital filters for given specifications.

CO-PO-PSO MAPPING MATRIX

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO₁	PSO₂	PSO₃
CO1	3	-	-	-	3	-	-	-	-	2	-	2	-	3	2
CO2	-	3	-	-	3	-	-	-	-	2	-	2	-	3	2
CO3	-	-	3	-	3	-	-	-	-	2	-	2	-	3	2
CO4	-	-	3	-	3	-	-	-	-	2	-	2	-	3	2



DAYANANDA SAGAR COLLEGE OF ENGINEERING DEPARTMENT OF ELECTRONICS & INSTRUMENTATION ENGINEERING

DO's

- Adhere and follow timings, proper dress code with appropriate foot wear.
- Bags, and other personal items must be stored in designated place.
- Come prepare with the viva, procedure, and other details of the experiment.
- Secure long hair, loose clothing & know safety and emergency procedures.
- Do check for the correct ranges/rating and carry one meter/instrument at a time
- Inspect all equipment/meters for damage prior to use
- Conduct the experiments accurately as directed by the teacher.
- Immediately report any sparks/ accidents/ injuries/ any other untoward incident to the faculty /instructor.
- Handle the apparatus/meters/computers gently and with care
- In case of an emergency or accident, follow the safety procedure.
- Switch OFF the power supply after completion of experiment

DONT's

- The use of mobile/ any other personal electronic gadgets is prohibited in the laboratory.
- Do not make noise in the Laboratory & do not sit on experiment table.
- Do not make loose connections and avoid overlapping of wires
- Don't switch on power supply without prior permission from the concerned staff.
- Never point/touch the CRO/Monitor screen with the tip of the open pen/pencil/any other sharp object.
- Never leave the experiments while in progress.
- Do not insert/use pen drive/any other storage devices into the CPU.
- Do not leave the Laboratory without the signature of the concerned staff in observation book

Experiment No: 01

Aim: Develop the Program to generate continuous waveforms and perform basic operations signals on MATLAB IDE.

Tools Required:

MATLAB IDE

MATLAB PROGRAM

1a) %Generate a 50Hz square wave using MATLAB function.

```
clc;
clear all;
%Enter the time vector
t=0:0.001:0.625;
x=square (2*pi*50*t);
plot(t, x)
title( 'square wave pulse');
xlabel ('time');
ylabel ('Amplitude');
```

Out Put: After execution of the program graphical window displays 50Hz Analog Square Wave.

1b) %Generate saw tooth, exponential, sine, and cosine waveforms employing MATLAB Signal Processing Commands.

```
clc;
clear all;
fs=120;
t=0:1/fs:1;
y=sawrooth (2*pi*40*t);
subplot (2,2,1);
plot (t,y);
xlabel('t');
ylabel('y(t)');
```

```
title ('Sawtooth Signal');
t=0:1:10;
y=0.4.^t;
subplot(2,2,2);
plot(t,y);
xlabel('t');ylabel('y(t)');
title('Exponential signal');
t=0.01:pi;
y=sin(2*pi*t);
subplot(2,2,3);
plot(t,y);
xlabel('t');
ylabel('y(t)');
title('Sine signal');
t=0:0.01:pi;
y=cos(2*pi*t);
subplot(2,2,4);
plot(t,y);
xlabel('t');
ylabel('y(t)');
title('Cosine Signal');
```

Out Put: After execution of the program graphical window displays saw tooth, exponential, sine, and cosine waveforms.

Experiment No: 02

2a) Develop the Program to generate discrete waveforms and perform basic operations on signals on MATLAB IDE.

Tools Required:

MATLAB IDE

MATLAB PROGRAM

%Program to generate discrete wave forms-Impulse, step, ramp and discrete signal.

```
clc;
clear all;
t=-3:1:3;
y=[zeros(1,3),ones(1,1),zeros(1,3)];
subplot (2,2,1);
stem(t,y);
xlabel('t');
ylabel('y(t)');
title('unit Impulse Signal');
n=input('Enter the n value');
t=0:1:n-1;
y=ones(1,n);
subplot(2,2,2);
stem(t,y);
xlabel('t');
ylabel(y(t)');
title('Unit Step Signal');
y= [1 2 4 6 8];
subplot (2,2,3);
stem(y);
xlabel('t');
ylabel(y(t)');
```

```
title ('Discrete Signal');  
n=input ('Enter the length of ramp sequence');  
t=0:n;  
subplot(2,2,4);  
stem('t');  
xlabel('t');  
ylabel(y(t));  
title ('Ramp Signal');
```

Out Put: After execution of the program graphical window displays discrete wave forms-Impulse, step, ramp and discrete signal.

2b) consider any two sinusoidal input signals of your own and carry out the following basic operations of these two input signals:

1) Addition 2) Subtraction 3) Multiplication 4) Scaling

Tools Required: MATLAB IDE

MATLAB PROGRAM

% Program to perform the mathematical operations on two sinusoidal signals.

```
clc;  
clear all;  
n=0:0.01:pi;  
y=sin(2*pi*n)+sin(3*pi*n);  
stem(n, y);  
xlabel('n');  
ylabel('y[n]');  
title('Addition of two given input signals');  
z=sin(2*pi*n)-sin(3*pi*n);  
figure(2);  
stem(n,z);  
xlabel('n');
```

```
ylabel('z[n]');

title('Subtraction of two given input signals');
w=sin(2*pi*n).*sin(3*pi*n);
figure(3);
stem(n, w);
xlabel('n');
ylabel('w[n]');
title('Multiplication of two given input signals');
p=5*(sin (2*pi*n)+sin(3*pi*n));
figure(4);
stem(n, p);
xlabel('n');
ylabel('p[n]');
title('Scaling of addition of given two input signals');
```

Out Put: After execution of the program graphical window displays mathematical operations (Addition, Subtraction, Multiplication and scaling) on two sinusoidal signals.

Experiment No: 03

Develop the program to determine linear convolution using MATLAB IDE.

Tools Required: MATLAB IDE

MATLAB PROGRAM

% Program to perform the linear convolution of two discrete time sequences.

```
clc;
clear all;
x=input('Enter the value of x[n]');
h=input('Enter the value of h[n]');
y=conv(x,h);
disp(y);
figure(1);
stem(y);
title('Linear Convolution of Two discrete sequences');
xlabel('n');
ylabel('y[n]');
```

Out Put: After execution of the program graphical window and figure window displays the linear convolution of two discrete sequences.

```
x[n]=[ 1 2 3 4]; h[n]=[5 6 7 8];
y[n]=x[n]*h[n]={5,16,34,60,61,52,32}
```

Experiment No: 04

Develop the program to determine circular convolution using MATLAB IDE.

Tools Required: MATLAB IDE

MATLAB PROGRAM

%Program to perform the circular convolution of two discrete time periodic sequences.

```
clc;
clear all;
x=input('Enter the value of x[n]');
h=input('Enter the value of h[n]');
N=input('Enter the length of resulting vector');
y=cconv(x, h, N);
disp(y);
figure (1);
stem (y);
title('Circular Convolution of Two discrete time periodic sequences');
xlabel('n');
ylabel('y[n]');
```

Out Put: After execution of the program graphical window and figure window displays the circular convolution of two discrete time periodic sequences.

$x[n] = [1 \ 2 \ 3 \ 4]; h[n] = [5 \ 6 \ 7 \ 8];$

$y[n] = x[n] \otimes h[n] = \{66, 68, 66, 60\}$

Experiment No: 05

Develop the program to verify Sampling theorem (All three cases) using MATLAB IDE.

Tools Required: MATLAB IDE

MATLAB PROGRAM

% Program to verification of the sampling theorem for all three cases.

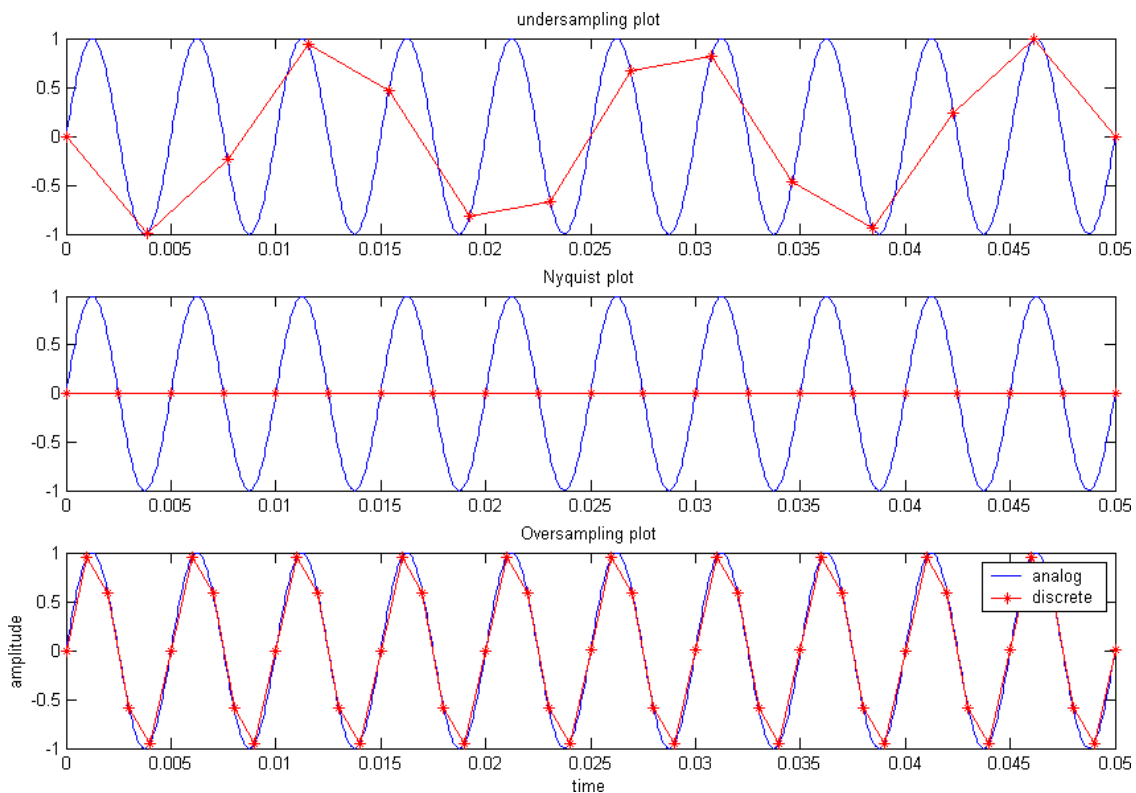
```
clc;
clear all;
tfinal=0.05;
t=0:0.00005:tfinal;
fd=input('Enter analog frequency');
%define analog signal for comparison
xt=sin(2*pi*fd*t);

%simulate condition for undersampling i.e.,  $fs1 < 2*fd$ 
fs1=1.3*fd;
%define the time vector
n1=0:1/fs1:tfinal;
%Generate the undersampled signal
xn=sin(2*pi*n1*fd);
%plot the analog & sampled signals
subplot(3,1,1);
plot(t,xt,'b',n1,xn,'r*-');
title('undersampling plot');
xlabel('time');
ylabel('amplitude');
legend('analog','discrete');

%condition for Nyquist plot
fs2=2*fd;
n2=0:1/fs2:tfinal;
xn=sin(2*pi*fd*n2);
subplot(3,1,2);
plot(t,xt,'b',n2,xn,'r*-');
title('Nyquist plot');
xlabel('time');
ylabel('amplitude');
legend('analog','discrete');
```



```
%condition for oversampling
fs3=5*fd;
n3=0:1/fs3:tfinal;
xn=sin(2*pi*fd*n3);
subplot(3,1,3);
plot(t,xt,'b',n3,xn,'r*-');
title('Oversampling plot');
xlabel('time');
ylabel('amplitude');
legend('analog','discrete');
```



Input: Enter the analog frequency: 500Hz

Out Put: After execution of the program graphical window and figure window displays the all three cases (1.Under sampling. 2. Nyquist Sampling 3.Over Sampling) of sampling theorem.

Experiment No: 06

Develop the program for N point DFT of a given discrete time sequence and to plot magnitude and phase spectrum using MATLAB IDE.

Tools Required: MATLAB IDE

% Program to find magnitude and phase of spectrum using FFT algorithm

```
clc;
clear all;
x=input('Enter the discrete time sequence');
y=fft(x);
disp(y);
M=abs(y);
disp(M);
theta=angle(y);
disp(theta);
figure(1);
stem(M);
title('Magnitude of Spectrum');
xlabel('n');
ylabel('magnitude');
figure(2);
stem(theta);
title('angle of spectrum');
xlabel('n');
ylabel('angle');
```

Out Put: After execution of the program graphical window and figure window displays the magnitude and phase of the spectrum.

$x[n] = [1 \ 2 \ 3 \ 4]$ $y = [10 \ -2+2i \ -2 \ -2-2i]$

$M = 10, 2.828, 2, 2.828$ θ (In radians) $= 0, 2.355, 3.14, -2.2828$

Experiment No: 07

Develop a program to determine linear convolution of two given discrete sequences using DFT and IDFT algorithms on MATLAB IDE.

Tools Required: MATLAB IDE

% Program to find the linear convolution using FFT and IFFT MATLAB commands.

```
clc;
clear all;
L=input('Enter the number of input elements');
M=input('Enter the number of impulse elements');
N=L+M-1;
x=input('Enter the value of x[n]');
h=input('Enter the value of h[n]');
x1=fft(x,N);
x2=fft(h,N);
z=x1.*x2;
y=ifft(z);
disp(y);
```

Out Put: After execution of the program graphical window and figure window displays the linear convolution of two discrete time sequences using DFT and IDFT algorithm.

```
x[n]=[ 1 2 3 4]; h[n]=[5 6 7 8];
y[n]=x[n]*h[n]={5,16,34,60,61,52,32}
```

Experiment No: 08

Develop a program to determine circular convolution of two given discrete time sequences using DFT and IDFT algorithms on MATLAB IDE.

Tools Required: MATLAB IDE

% Program to find the circular convolution using FFT and IFFT MATLAB commands.

```
clc;
clear all;
x=input('Enter the value of x[n]');
h=input('Enter the value of h[n]');
N=input('Enter the length of resulting vector');
y1=fft(x, N);
y2=fft(h, N);
y3=y1.*y2;
y=ifft(y3);
disp(y);
figure(1);
stem(y);
title('circular convolution using DFT and IDFT method');
xlabel('n');
ylabel('y[n]');
```

Out Put: After execution of the program graphical window and figure window displays the circular convolution of two discrete time sequences using DFT and IDFT algorithm.

$x[n] = [1 \ 2 \ 3 \ 4]; h[n] = [5 \ 6 \ 7 \ 8];$

$y[n] = x[n] \otimes h[n] = \{66, 68, 66, 60\}$

Experiment No: 09

Design and Develop the Program for FIR filter response using Rectangular and Hamming Window Technique using MATLAB IDE.

Tools Required: MATLAB IDE

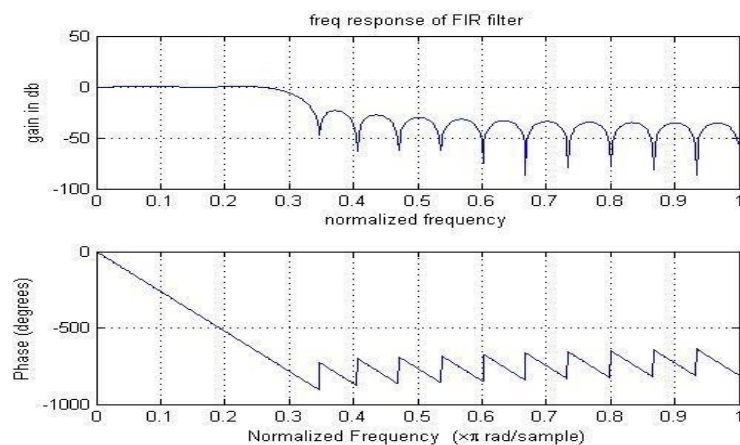
```
%MATLAB program for hamming window  
clc;  
clear all;  
n=input('Enter the length of the filter');  
wc=input('Enter the cut off frequency in radians');  
B=hamming(n);  
disp(B);  
b=fir1(n-1,wc/pi,B,'noscale');  
freqz(b,512,10000);  
title('Hamming Window');
```

Out Put: After execution of the program

Enter the value of n=10

Cut off frequency=0.2 radians/sample

Response of hamming window will display on figure window and hamming window coefficients will be displayed on the command window



Experiment No: 10

Design and Develop the Program for Butterworth IIR filter response using MATLAB IDE.

Tools Required: MATLAB IDE

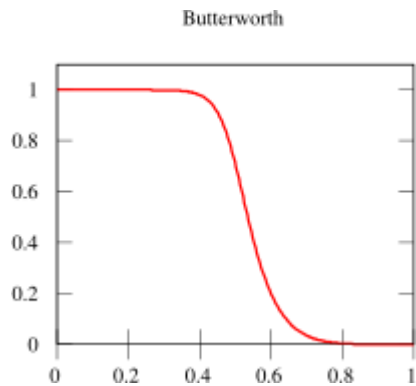
```
%MATLAB program for Butterworth filter
clc;
clear all;
n=input('Enter the order of the filter');
w=input('Enter the digital frequency in radians per sample=');
[b,a]=butter(n,w)
W=0:0.01:pi;
[h,am]=freqz(b,a,w);
ampd=20*log(abs(h));
an=angle(h);
subplot(2,1,1);
plot(w/pi, ampd);
grid on;
xlabel('normalized frequency=');
ylabel('gain in db=');
subplot(2,1,2);
plot(w/pi, an);
xlabel('normalized frequency=');
ylabel('angle in radians=');
```

Out Put: After execution of the program

Enter the value of $n=2$

Cut off frequency=0.2 radians/sample

Response of 2nd order Butterworth filter will display on figure window and butter worth filter coefficients will be displayed on the command window



C-PROGRAMS ON CC (CODE COMPOSER) STUDIO IDE- TMS3206713 TARGET BOARD

General Execution Procedure for execution of all programs on CC STUDIO IDE

1. Open Code Composer Studio; make sure the DSP kit is turned on.
2. Start a new project using 'Project-new ' pull down menu, save it in a Separate directory(C:\ti\myprojects) with name demo.pjt.
3. Add the source files demo.c to the project using 'Project→add files to project' pull down menu.
4. Add the linker command file hello.cmd
(Path: \ti\tutorial\dsk6713\hello1\hello.cmd)
5. Add the run time support library file rts6700.lib
(Path: c:\ti\c6000\cgtools\lib\rts6700.lib)
6. Compile the program using the 'Project-compile' pull down menu or by clicking the shortcut icon on the left side of program window.
7. Build the program using the 'Project-Build' pull down menu
8. Load the program(demo.out) in program memory of DSP chip using the 'File-load program' pull down menu.
9. To View output graphically
 - Select view → graph → time and frequency.
10. Interface the hardware DSP 6713 Target Board to the Personal Computer and observe the Personal Computer.

Experiment No: 01

Develop the C program to verify Sampling theorem (All three cases) using CC studio IDE.

Tools Required: PC loaded with CC Studio IDE, TMS320-DSPK6713 Target Board.

C Program

```
#include<stdio.h>
#include<math.h>
float x[40], y[40], z[40];
main ()

{

    int i;

    //generate 40 samples of required frequency

    for (i=0; i<40; i++)

    {

        //Under sampled signal of 500Hz with sampling frequency of 800Hz

        x[i]=sin(2*3.14*i*500/800);

        //Nyquist sampled signal of 500Hz with sampling frequency of 1000Hz

        y[i]= sin(2*3.14*i*1500/1000);

        /Over sampling signal of 700Hz with sampling frequency of 1000Hz

        z[i]= sin(2*3.14*i*1500/1500);
    } //end of for loop

}
```

Out Put: Sampling theorem was verified for under sampling, Nyquist sampling and over sampling. Results were observed on graphical window.

Experiment No: 02

Develop the C program to determine the linear convolution of the two discrete time sequences.

Tools Required: PC loaded with CC Studio IDE, TMS320-DSPK6713 Target Board.

C Program (Linear Convolution.c)

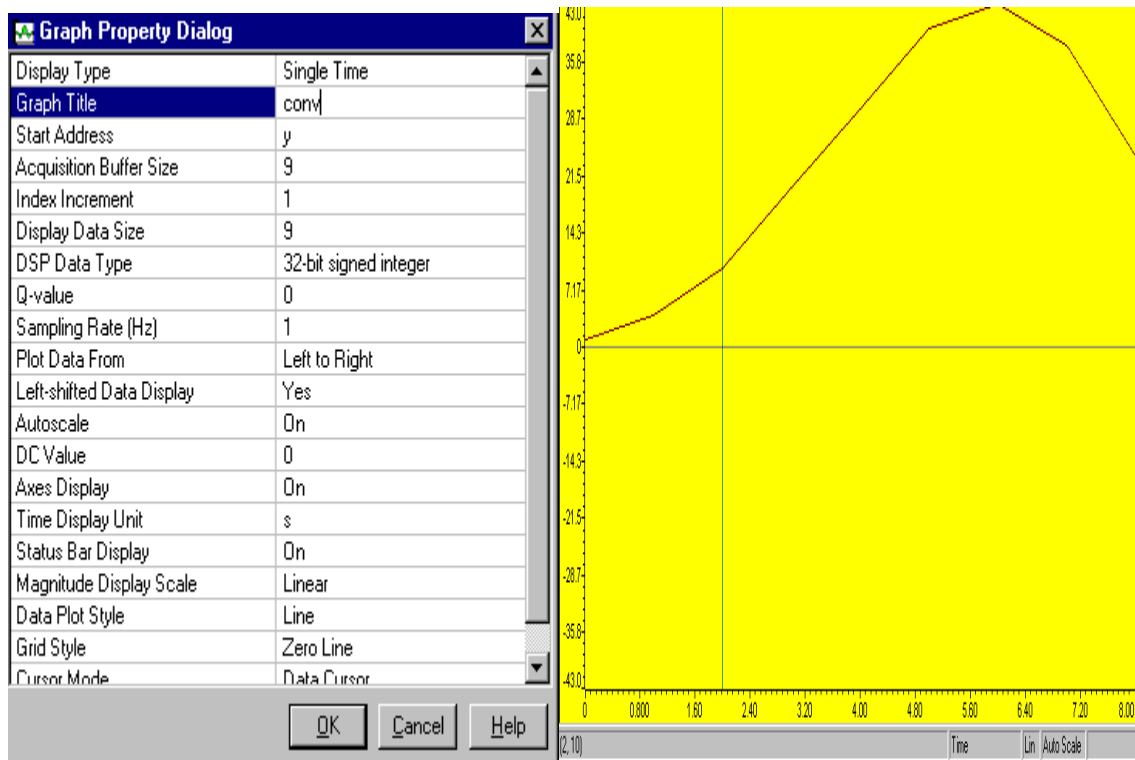
```
#include<stdio.h>
#include<math.h>
int x[10],h[10],y[10];
main()
{
    int i,j,k;
    int xlen,hlen,ylen;
    printf("Enter the input sequence length=");
    scanf("%d",&xlen);
    printf("Enter the impulse sequence length=");
    scanf("%d",&hlen);
    ylen=(xlen+hlen)-1;
    printf("Enter the input sequence element:\n");
    for(i=0;i<xlen;i++)
    {
        printf(("x[%d]=",i);
        scanf("%d",&x[i]);
    }
```

```
printf("Enter the impulse sequence element:\n");
for(i=0;i<hlen;i++)
{
    printf(("h[%d]=",i);
    scanf("%d",&h[i]);
}
printf("Out put sequence elements:\n");
for(i=0;i<ylen;i++)
y[i]=0;
for(k=0;k<ylen;i++)
{
    for(i=k,j=0;;i--,j++)
    {
        if((i<0||j>hlen-1))
            break;
        if(i>xlen-1)
            continue;
        y[k]+=x[i]*h[j];
    }
    printf("y[%d]=%d\n",k,y[k]);
}
}
```

Out Put: After execution of the program graphical window displays the linear convolution of two discrete time sequences.

```
x[n]=[ 1 2 3 4]; h[n]=[5 6 7 8];
y[n]=x[n]*h[n]={5,16,34,60,61,52,32}
```

LINEAR CONVOLUTION GRAPHICAL OUT PUT ON GRAPHICAL WINDOW



Experiment No: 03

Develop the C program to determine the circular convolution of the two discrete time sequences.

Tools Required: PC loaded with CC Studio IDE, TMS320-DSPK6713 Target Board.

C Program (Circular Convolution.c)

```
#include<stdio.h>
#include<math.h>
float x[10],h[10],y[10];
main()
{
    int i,k,n;
    int xlen,hlen,ylen;
    printf("Enter the input sequence length=");
    scanf("%d",&xlen);
    printf("Enter the impulse sequence length=");
    scanf("%d",&hlen);
    ylen=xlen
    printf("Enter the input sequence element:\n");
    for(i=0;i<xlen;i++)
    {
        printf("x[%d]=",i);
        scanf("%f",&x[i]);
        printf("\n");
    }
```

```
printf("Enter the impulse sequence element:\n");
```

```
for(i=0;i<hlen;i++)
```

```
{
```

```
    printf(("h[%d]=",i);
```

```
    scanf("%f",&h[i]);
```

```
    printf("\n");
```

```
}
```

```
for(n=0;n<ylen;n++)
```

```
y[n]=0;
```

```
for(k=0;k<ylen;k++)
```

```
{
```

```
i=(n-k)%ylen;
```

```
if(i<0)
```

```
i=i+ylen;
```

```
y[n]+=x[k]*h[i];
```

```
}
```

```
printf("y[%d]=% f \n",n,y[n];
```

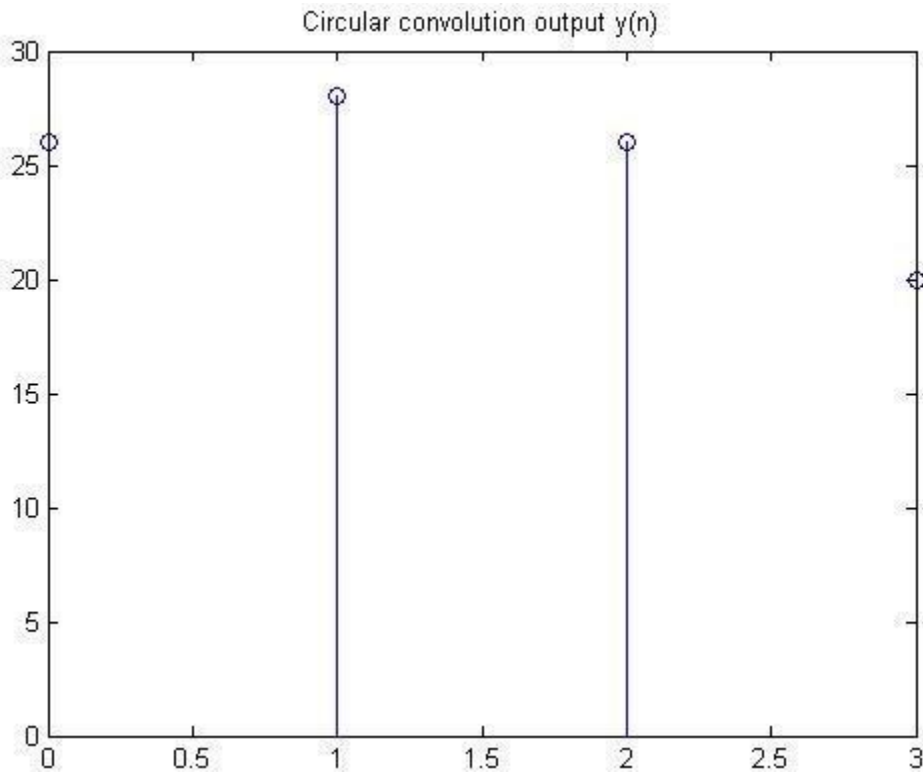
```
}
```

```
}
```

Out Put: After execution of the program graphical window displays the circular convolution of two discrete time sequences.

$x[n] = [1 \ 2 \ 3 \ 4]$; $h[n] = [5 \ 6 \ 7 \ 8]$;

$y[n] = x[n] \otimes h[n] = \{66, 68, 66, 60\}$



Experiment No: 04

Develop the C program for N point DFT of a given discrete time sequence.

Tools Required: PC loaded with CC Studio IDE, TMS320-DSPK6713 Target Board.

C Program (dft.c)

```
#include<stdio.h>
#include<math.h>
int main()
{
    int N,k,n;
    float static X[100],X_real[100],X_Imag[100];
    printf("\t Discrete Fourier Transform (DFT)\n");
    scanf("%d",&N);
    printf("\nEnter the sequence x[n]\n");
    for(n=0;n<N;n++)
    {
        printf("X(%d)=",n);
        scanf("%f",&X[n]);
    }
    for(k=0;k<N;k++)
    {
        X_real[k]=X_Imag[k]=0.0;
        for(n=0;n<N;n++)
        {
            X_real[k]=X_real[k]+x[n]*cos((2*3.1416*k*n))/N;
            X_Imag[k]=X_Imag[k]-x[n]*sin((2*3.1416*k*n))/N;
        }
    }
}
```



```
for(k=0;k<N;k++)  
    printf("X(%d)=\t%f\t+%f i\n",k, X_real[k], X_imag[k]);  
return(0);  
}
```

Out Put: After execution of the program

Enter the value of N=4

$x[n] = [1 \ 2 \ 3 \ 4]$

DFT of $x[n]$ is $X[k] = [10 \ -2+2i \ -2 \ -2-2i]$

Experiment No: 05

Design and Develop the C program for FIR filter response.

Tools Required: PC loaded with CC Studio IDE, TMS320-DSPK6713 Target Board.

C Program (fir.c)

```
#include<stdio.h>

#include<math.h>

#define pi 3.14

float x[100],y[140];

float h[11]={0.080,0.1679,0.3979,0.6821,0.9121,1.000,0.9121,0.6821,0.33979,0.1679,0.0800};

int xlength=60,hlength=20,N;

main()
{
    int i,n,k;

    for(i=0;i<20;i++)
        x[i]=sin(2*pi*i*50/1000);

    for(i=20;i<40;i++)
        x[i]=sin(2*pi*i*350/1000);

    for(i=40;i<60;i++)
        x[i]=sin(2*pi*i*250/1000);

    N=(xlength+hlength)-1;

    for(n=0;n<N;n++)
    {
        y[n]=0;

        for(k=0;k<xlength;k++)
        {
            if((n-k)>=0)&((n-k)<hlength)
                y[n]=y[n]+x[k]*h[n-k];
        }
    }
}
```

```
}
```

```
}
```

```
}
```

Out Put: After execution of the program FIR filter response for the given input displayed on the graphical window.

Experiment No: 06

Design and Develop the C program for IIR filter response.

Tools Required: PC loaded with CC Studio IDE, TMS320-DSPK6713 Target Board.

C Program (fir.c)

```
#include<stdio.h>

#include<math.h>

#define pi 3.14

float b0=0.0675,b1=0.1349,b2=0.0675;

float a0=0.4128,,a1=-1.1430,a2=1;

float dn=0,dn1=0,dn2=0;

int n=60;

float x[60],y[70];

main()

{

int k,i;

for(i=0;i<20;i++)

x[i]=sin(2*pi*i*100/2140);

for(i=20;i<40;i++)

x[i]=sin(2*pi*i*700/2140);

for(i=40;i<60;i++)

x[i]=sin(2*pi*i*50/2140);

for(k=0;k<n;k++)

{

y[k]=b0*dn+b1*dn1+b2*dn2;

dn2=dn1;

dn1=dn;

dn=x[k]-a1*dn1-a2*dn2;
```

```
}
```

```
}
```

Out Put: After execution of the program IIR filter response for the given input displayed on the graphical window.